COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S4
DATE PERFORMED:	August 28 2026	**DATE SUBMITTED:**	Aug 28, 2026
NAME:	Cardenas, Lea Marie N.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 6 SEARCHING TECHNIQUES

1. PROCEDURE OUTPUT

Copy and paste the questions from the lab and include your answers after.

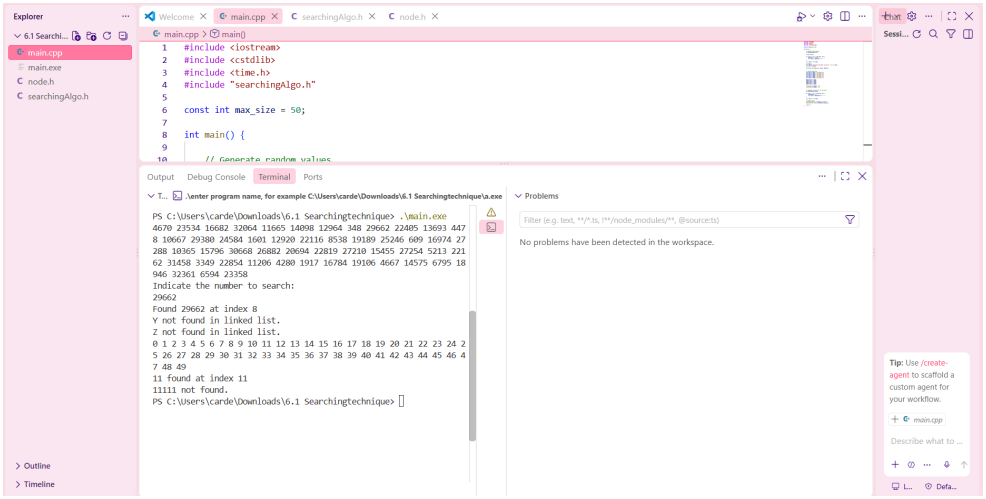
Screenshot	
Observation	In this activity I started by creating the C++ program. I generated 50 random numbers and stored them in an array, then used linear search to find the entered number by the user (like in this picture).

Table 6-1. Data Generated and Observations.

Code

```
#include <iostream>
#include <cstdlib> // for generating random
#include <time.h>
#include "node.h"
#include "searchingAlgo.h"

const int maxSize = 50;

int main(){
//ARRAY LINEAR SEARCH TEST
    //generate
    int dataset[maxSize];
    srand(time(0));
    for(int i = 0; i < maxSize; i++){
        dataset[i] = rand();
    }

    for (int i = 0; i < maxSize; i++){
        std::cout << dataset[i] << " ";
    }
    std::cout << std::endl;

    int userInput;
    std::cout << "What number to search?";
    std::cin >> userInput;
    std::cout << "Array result LS: ";
    linearSearch(maxSize, userInput, dataset);

    std::cout << "What number to search?";
    std::cin >> userInput;
    std::cout << "Array result LS: ";
    linearSearch(maxSize, userInput, dataset);
```

Output

```
Terminal

PS C:\Users\carde\Downloads\6.1 Searchingtechnique> g++ main.cpp -
o main.exe
PS C:\Users\carde\Downloads\6.1 Searchingtechnique> .\main.exe
29334 18110 5634 21613 9778 5315 15526 19626 7746 6780 12536 29045
 29664 29168 13563 3223 23098 4063 32632 31364 31114 748 3733 664
30280 11778 2217 1185 30877 496 8015 8094 18460 31318 10426 13766
20397 25029 25416 29132 7625 8482 5977 12642 13549 7755 2913 30972
 16456 28157
What number to search?20397
Array result LS: Searching is successful
What number to search?29045
Array result LS: Searching is successful
```

Observation

The linear search checked the array sequentially starting from index 0 and required 11 comparisons to locate the target value at index 10, confirming that the number of comparisons grows in direct proportion to the position of the element.

Table 6-2a. Linear Search for Arrays

Code

```
'/LINKED LIST LINEAR SEARCH TEST
    Node<char> *name1 = new_node('M');
    Node<char> *name2 = new_node('A');
    Node<char> *name3 = new_node('R');
    Node<char> *name4 = new_node('I');
    Node<char> *name5 = new_node('E');

    name1->next = name2;
    name2->next = name3;
    name3->next = name4;
    name4->next = name5;

    Node<char>* current = name1;
    while (current != nullptr) {
    std::cout << current->data << " ";
    current = current->next;
    }
    std::cout << "\n";

    char charInput;
    std::cout << "What character to search?";
    std::cin >> charInput;
    std::cout << "Linked list result LS: ";
    linearSearchLL(name1, charInput);

    std::cout << "What character to search?";
    std::cin >> charInput;
    std::cout << "Linked list result LS: ";
    linearSearchLL(name1, charInput);
```

Output

```
M A R I E
What character to search?Y
Linked list result LS: Searching is unsuccessful
What character to search?Z
Linked list result LS: Searching is unsuccessful
3 5 6 8 11 12 14 15 17 18
```

Observation

The linked list contains the characters M, A, R, I, E, which are displayed sequentially by traversing the nodes. When I enter a character to search, the linearSearchLL() function checks each node one by one. In the shown output, the characters Y and Z are not present in the linked list, so the program displays “Searching is unsuccessful.”

Table 6-2b. Linear Search for Linked List

Code

```
//ARRAY BINARY SEARCH TEST
    int sortedArr[] = {3, 5, 6, 8, 11, 12, 14, 15, 17, 18};
    int arrSize = 10;

    for (int i = 0; i < arrSize; i++) {
        std::cout << sortedArr[i] << " ";
    }
    std::cout << "\n\n";

    // CASE 1: Successful
    std::cout << "Case 1: Search for 8\n";
    binarySearchArr(arrSize, 8, sortedArr);

    // CASE 2: Unsuccessful - smaller than smallest
    std::cout << "Case 2: Search for 2\n";
    binarySearchArr(arrSize, 2, sortedArr);

    // CASE 3: Unsuccessful - larger than largest
    std::cout << "Case 3: Search for 20\n";
    binarySearchArr(arrSize, 20, sortedArr);
```

Output

```
Case 1: Search for 8
Searching is successful
Case 2: Search for 2
Searching is unsuccessful
Case 3: Search for 20
Searching is unsuccessful
3 5 6 8 11
```

Observation

Its demonstrates binary search using an array by searching for values in a sorted list. It successfully finds 8 because it is present in the array, while the searches for 2 and 20 are unsuccessful because they are outside the range of the array values.

Table 6-3a. Binary Search for Arrays

Code	
	``
	//LINKED LIST BINARY SEARCH TEST
	    Node<int>* head = new_node(3);
	    Node<int>* node2 = new_node(5);
	    Node<int>* node3 = new_node(6);
	    Node<int>* node4 = new_node(8);
	    Node<int>* node5 = new_node(11);
	
	    head->next = node2;
	    node2->next = node3;
	    node3->next = node4;
	    node4->next = node5;
	
	    Node<int>* currentInt = head;
	while (currentInt != nullptr) {
	    std::cout << currentInt->data << " ";
	    currentInt = currentInt->next;
	}
	    std::cout << "\n";
	
	    // CASE 1: Successful
	    std::cout << "Case 1: Search for 8\n";
	    binarySearchLL(head, 8);
	    // CASE 2: Unsuccessful - smaller than smallest
	    std::cout << "Case 2: Search for 2\n";
	    binarySearchLL(head, 2);
	    // CASE 3: Unsuccessful - larger than largest
	    std::cout << "Case 3: Search for 15\n";
	    binarySearchLL(head, 15);
	
	    return 0;
	}
	``

output	 ``` Searching is unsuccessful 3 5 6 8 11 Case 1: Search for 8 Searching is successful Case 2: Search for 2 Searching is unsuccessful Case 3: Search for 15 Searching is unsuccessful PS C:\Users\carde\Downloads\6.1 Searchingtechnique> ```
Observation	It demonstrates how binary search works on a sorted linked list. It contains the values 3, 5, 6, 8, and 11, which are arranged in ascending order. The program tests different search cases: 8 is successfully found because it exists in the list, while 2 and 15 are unsuccessful because they are smaller than the smallest value and larger than the largest value, respectively. This shows that binary search can quickly determine whether a value exists by narrowing down the possible search area instead of checking every value one by one.

Table 6-3b. Binary Search for Linked List

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

SUPPLEMENTARY.

For each provided problem, give a screenshot of your code, the output console, and your answers to the questions.

Problem 1. Suppose you are doing a sequential search of the list [15, 18, 2, 19, 18, 0, 8, 14, 19, 14]. Utilizing both a linked list and an array approach to the list, use sequential search and identify how many comparisons would be necessary to find the key '18'?

Code

```
#include <iostream>
#include "node.h"
#include "searchingAlgo.h"

int main() {

    // ========================================
    // PROBLEM 1
    // Sequential Search using Array and Linked List
    // ========================================

    int dataFind = 18;

    // ARRAY APPROACH
    std::cout << "===== PROBLEM 1 =====\n";
    std::cout << "ARRAY APPROACH\n";

    int listOne[] = {15, 18, 2, 19, 18, 0, 8, 14, 19, 14};
    int size = 10;

    int comparisons = 0;

    for (int i = 0; i < size; i++) {

        comparisons++;

        std::cout << "Comparison " << comparisons
                  << ": " << listOne[i]
                  << " vs " << dataFind << "\n";

        if (listOne[i] == dataFind) {
            break;
        }
    }
```

```
std::cout << "Total comparisons: "
          << comparisons << "\n\n";


// LINKED LIST APPROACH
std::cout << "LINKED LIST APPROACH\n";

Node<int>* head = new_node(15);
Node<int>* node2 = new_node(18);
Node<int>* node3 = new_node(2);
Node<int>* node4 = new_node(19);
Node<int>* node5 = new_node(18);
Node<int>* node6 = new_node(0);
Node<int>* node7 = new_node(8);
Node<int>* node8 = new_node(14);
Node<int>* node9 = new_node(19);
Node<int>* node10 = new_node(14);

// Connect the nodes
head->next = node2;
node2->next = node3;
node3->next = node4;
node4->next = node5;
node5->next = node6;
node6->next = node7;
node7->next = node8;
node8->next = node9;
node9->next = node10;


// Display linked list
Node<int>* current = head;

while (current != nullptr) {
    std::cout << current->data << " ";
    current = current->next;



 // Display linked list
 Node<int>* current = head;

 while (current != nullptr) {
     std::cout << current->data << " ";
     current = current->next;
 }

 std::cout << "\n";


 // Sequential search in linked list
 comparisons = 0;
 current = head;

 while (current != nullptr) {

     comparisons++;

     std::cout << "Comparison " << comparisons
               << ": " << current->data
               << " vs " << dataFind << "\n";

     if (current->data == dataFind) {
         break;
     }

     current = current->next;
 }

 std::cout << "Total comparisons: "
           << comparisons << "\n\n";
```

Output	ARRAY APPROACH Comparison 1: 15 vs 18 Comparison 2: 18 vs 18 Total comparisons: 2 LINKED LIST APPROACH 15 18 2 19 18 0 8 14 19 14 Comparison 1: 15 vs 18 Comparison 2: 18 vs 18 Total comparisons: 2
Observation	The program shows that sequential search works in both an array and a linked list, and the value 18 is successfully found after 2 comparisons in both approaches.

Problem 2. Modify your sequential search algorithm so that it returns the count of repeating instances for a given search element 'k'. Test on the same list given in problem 1.

Code	`std::cout << "===== PROBLEM 2 =====\n";`
	
	`int count = countOccurrences(listOne, size, 18);`
	
	`std::cout << "Number of occurrences of 18: "`
	`<< count << "\n\n";`

Output	===== PROBLEM 2 ===== Number of occurrences of 18: 2
Observation	The program successfully counts the number of times 18 appears in the array and determines that it occurs 2 times.

Problem 3. Suppose you have the following sorted list [3, 5, 6, 8, 11, 12, 14, 15, 17, 18] and are using the binary search algorithm. If you want to find the key 8, draw a diagram that shows how the searching works per iteration of the algorithm. Prove that your drawing is correct by implementing the algorithm and showing a screenshot of the code and the output console.

Code	```
	// ==
	// PROBLEM 3
	// Iterative Binary Search
	// ==
	
	std::cout << "===== PROBLEM 3 =====\n";
	
	int listTwo[] = {
	    3, 5, 6, 8, 11,
	    12, 14, 15, 17, 18
	};
	
	int sizeTwo = 10;
	
	std::cout << "Searching for 8...\n";
	
	binarySearchDemo(listTwo, sizeTwo, 8);
	
	std::cout << "\n";
	``

Output	``` ===== PROBLEM 3 ===== Searching for 8... Searching is successful ```
Observation	The program uses iterative binary search on a sorted array and successfully finds the value 8 by repeatedly reducing the search area.

Problem 4. Modify the binary search algorithm so that the algorithm becomes recursive. Using this new recursive binary search, implement a solution to the same problem for problem 3.

Code	```
	// ==
	// PROBLEM 4
	// Recursive Binary Search
	// ==
	
	std::cout << "===== PROBLEM 4 =====\n";
	
	int listThree[] = {
	    3, 5, 6, 8, 11,
	    12, 14, 15, 17, 18
	};
	
	int sizeThree = 10;
	
	std::cout << "Searching for 8...\n";
	
	bool found = binarySearchRecursive(
	    listThree,
	    0,
	    sizeThree - 1,
	    8
	);
	
	if (found) {
	    std::cout << "Searching is successful\n";
	}
	else {
	    std::cout << "Searching is unsuccessful\n";
	}
	``

Output	``` ===== PROBLEM 4 ===== Searching for 8... Searching is successful ```
Observation	The program uses recursive binary search to search for 8, and it successfully finds the value, showing that recursion can also be used to perform binary search.

Conclusion

Through this coding procedure I learned how to do both array and linked lists full of procedures and supplementary. But I had problems in problem 3 and 4 but it shows in the code what should be the output.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT and Grammarly**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **I used chatgpt for my debugging and I used grammarly to check my sentences grammar**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I used ChatGPT only as a guide for debugging and improving my code, the problems were done by me and the pdf guidelines. Grammarly was used only to check my grammar and clarity of my written explanations. AI tools did not replace my original works.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.